# CodeInspector: Project Requirements

Initial baseline: 0.1.0 | 19 October 2026

Course: Software Construction and Development, IMSciences.

Instructor: Engr. Muhammad Umer Haroon.

## 1. Project Member

Abdul Samad | BSSE, Group A | Semester 5 | Roll number 2.

GitHub username: abdulsamad-codes.

Repository: https://github.com/abdulsamad-codes/CodeInspector

This is an individual project with instructor permission, as confirmed by Abdul Samad. He is responsible for requirements, development, testing, documentation, Git history, and submission.

## 2. Problem, Objectives, and Users

Python student projects may contain unclear names, inconsistent formatting, large functions, repeated code, and incomplete documentation. Manual review can take time and miss issues.

CodeInspector will inspect a local Python project and provide specific findings with locations and improvement suggestions. Its objectives are to support clean code practice, apply selected PEP 8 guidance and course rules, and make project review easier. Intended users are students and instructors.

Sir Umer recommended researching PEP 8 after reviewing the project idea. PEP 8 is a style guide, not a Python module. The project will implement a selected, documented subset of its guidance rather than claim to check every recommendation.

## 3. Functional Requirements

### FR-01: Read a Python project [Must]

Accept a local folder path, find Python files recursively, and exclude Git data, virtual environments, and generated/cache folders. Do not follow links outside the selected project. Report an invalid path or a project with no Python files clearly.

Check: A sample folder produces the expected file list while excluded files are ignored.

### FR-02: Analyze without executing code [Must]

Read source using its declared Python encoding and parse structure using the built-in ast module. Use source text and tokens for layout checks that AST does not preserve. Do not run or import the submitted code. Report syntax/read errors and continue with other readable files.

Check: An input containing a top-level file-writing statement does not execute. A syntax error in one file does not hide results from valid files.

### FR-03: Check selected PEP 8 naming conventions [Must]

Check lowercase underscore-separated function/variable names, CapWords class names, lowercase module/package names, and uppercase module-level constants where they can be identified conservatively. Check conventional self/cls receiver names and warn about ambiguous single-letter names l, O, and I. Respect documented special names and underscore conventions.

Check: Examples cover conforming and nonconforming names, methods, special methods, and constants. Findings explain their rule and location. Python does not declare constants explicitly, so uncertain classifications require review.

### FR-04: Check selected PEP 8 layout conventions [Must]

Check four-space block indentation, code lines over 79 characters, comment/docstring lines over 72, definition spacing, avoidable whitespace, and simple import layout. Continuation lines and documented exceptions must be handled separately. Start with checks that can be tested reliably; list supported checks in the report/documentation.

Check: Fixtures include correctly formatted code, each supported violation, and valid multiline statements. Configurable line limits allow project-specific conventions.

### FR-05: Apply course clean code rules [Must]

Review meaningful names, function size, nesting depth, parameter counts, multiple returns, loop break/continue, and direct recursion. Where feasible, distinguish a direct tail-call pattern from a non-tail pattern. These are course review rules, not PEP 8 rules or proof of incorrect code.

Initial configurable defaults: 20 physical lines per function, nesting depth 2 with the function body counted as level 1, and 2 explicit parameters excluding self/cls. The 20-line limit is a project decision; the slides ask for small functions without specifying a line count. Zero/one explicit return is acceptable for the multiple-return check. Nested functions are evaluated separately.

Check: Test boundaries at and above each limit, nested functions, direct recursion, and return patterns. Explain that static checks cannot prove a function has one responsibility or resolve every recursive call.

### FR-06: Check project completeness [Must]

Identify missing root README files, missing conventional test files, and empty functions. Accept README, README.md, README.rst, or README.txt case-insensitively. Recognize test_*.py and *_test.py outside excluded folders. Empty functions include docstring-only, pass-only, and ellipsis-only bodies.

Check: Examples cover present/absent files and implemented/empty functions. State that file presence does not establish documentation quality or test coverage.

### FR-07: Configure rules and report findings [Must]

Provide documented defaults and a settings file for supported limits and enabled rules. Reject invalid settings clearly. Produce a readable terminal report with rule identifier, category, relative path, line number where applicable, explanation, and suggestion. Distinguish PEP 8, course, and completeness findings. Project-level findings have no invented line number. Use stable ordering and clearly mark incomplete scans.

Check: Disabling a rule removes its findings; invalid limits produce useful errors; repeated scans produce the same ordered findings for the same input/settings.

### FR-08: Extend analysis and scoring [Should]

After the core checks work, add conservative unused-import analysis, candidate duplicate-function detection, report export, and category/overall scoring. Define score penalties before implementing them; disabled rules must not reduce scores, and scan failures must not appear as a perfect complete scan.

Check: Used/unused imports and similar/different functions have documented examples. Reported scores agree with hand-calculated cases.

**FR-09: Research AI-use detection [Could: later research]**

Investigate possible AI-associated style signals after the core analyzer is stable. Research must examine false positives and limitations before deciding whether a feature is suitable. No working detector or reliable accuracy is promised at this stage. Any future estimate must be labelled experimental and must not be presented as proof of authorship or misconduct.

Check before inclusion: Document the method, evaluation data, false-positive results, limitations, and how uncertainty is communicated. Avoid presenting an unvalidated percentage as a reliable probability.

## 4. Non-Functional Requirements

- Safety/privacy: Analyze locally without executing target code or uploading it. Do not modify input source files.
- Maintainability: Keep parsing, rule checks, settings, and reporting separate. Use descriptive names, focused functions, and consistent conventions.
- Reliability: Handle invalid paths, unreadable files, syntax errors, and invalid settings with useful messages.
- Reproducibility: Keep findings ordered and identify enabled checks and the analyzer version in reports.
- Portability: Target Python 3.11 or newer initially; avoid machine-specific drive paths in application code.
- Usability: Document actual commands, supported checks, configuration, and limitations as they are implemented.
- Verification: Test normal inputs, rule boundaries, valid exceptions, error handling, and the no-execution guarantee.

Initial inputs are small student repositories. Performance limits and benchmarks will be decided during implementation; no measured performance claim is made yet.

## 5. Priorities and Development Plan

P0 - Requirements, README, backlog, naming/version conventions, and the first Git baseline.

P1 - File discovery and safe parsing, followed by PEP 8 naming/layout checks, course rules, completeness checks, settings, and reporting (FR-01 to FR-07).

P2 - Unused imports, duplicate-code review, exports, and transparent scoring (FR-08).

P3 - AI-use detection research and feasibility review (FR-09).

Abdul Samad owns all tasks. Detailed status is kept in docs/backlog.md. Complete work in small, testable pieces and update requirements as understanding changes. There are no invented implementation deadlines.

## 6. Assumptions, Constraints, and Limitations

- Users provide a readable local Python project. Syntax support depends on the running Python interpreter.
- The initial interface is a command-line tool. A database, cloud service, and web interface are outside initial scope.
- PEP 8 style compliance does not prove meaningful names, correct logic, security, or good architecture. Some guidance needs context or human judgment.
- Short/generic-name heuristics come from the course/project, not a PEP 8 ban on every short name.
- Function-length, argument-count, return-count, and recursion rules are separate course/project guidance. Python does not optimize tail recursion.
- Test-file detection cannot measure test coverage. Import/similarity analysis may miss dynamic behavior and produce false positives.

- Scores summarize configured checks and are not industry-readiness certificates.
- AI-use detection remains research; style alone cannot establish authorship. Cross-team plagiarism detection is outside scope.
- Scope is limited by the semester and one student's capacity. Optional work follows a tested core.

## 7. Naming, Versioning, and Review

Python naming follows the selected PEP 8 conventions in FR-03. Folders and ordinary documents use descriptive lowercase names; standard README/CHANGELOG names and supplied PDF names are preserved.

Use meaningful commits for actual chunks of work, with Git history instead of duplicate versioned filenames. Use annotated vMAJOR.MINOR.PATCH tags for significant milestones. v0.1.0 identifies the planning/requirements baseline, not a completed analyzer. Later 0.x milestones may change developing interfaces; v1.0.0 will represent the agreed complete version. CHANGELOG.md records each milestone.

Requirements are maintained in docs/requirements.md. The matching docs/requirements.pdf is the submission copy and must be updated with its source. The document baseline may remain unchanged for unrelated implementation changes.

Before submission, Abdul Samad must review and understand the documents, verify unauthenticated public repository access, and submit the repository URL plus the exact committed PDF. The assignment warns against copied or massively AI-generated work and allows a viva; the student must follow the instructor's assistance rules and be able to explain the submitted work.

## 8. References


- Assignment_1_SCD_IMSCIENCES.pdf, pages 2-3: required repository, requirements content, responsibilities, history/tags, and submission.
- CodeInspector_Project_Proposal.pdf, page 1: initial project scope and phases. This document refines the proposal with PEP 8 and records AI-use detection as later research.
- Official PEP 8: https://peps.python.org/pep-0008/ (reviewed 19 October 2026).
- Lecture 1, pages 11 and 14-16: construction principles and meaningful names.
- Lecture 2, pages 9-13 and 39-43: reuse, small functions, naming, and shallow nesting.
- Lecture 2b, pages 7-11 and 17-26: arguments, entry/exit guidance, recursion, and tail calls.
- Lecture 3, pages 7-12 and 18-28: stack choice, releases, changelog, and semantic versioning.
- Lecture 4, pages 4-14 and 37-43: regex, raw strings, and text patterns. Python structure will use AST rather than regex alone.


Lecture filenames correspond to the supplied Lectures folder in the parent course directory. These sources guide the project; application code will not depend on that local directory.